

STARTER KIT

What's inside, how it works, and how to set it up

A pre-configured `.claude/` folder that gives you a professional Claude Code setup in 30 seconds.

What is the Starter Kit?

When you run Claude Code in a project, it looks for a `.claude/` folder. If it finds one, it loads the config inside: project rules, custom commands, safety hooks, and more.

Without it, Claude starts from scratch every time. With it, Claude knows your project, follows your rules, and has custom commands ready. Think of it as Claude's manual for YOUR specific project. This Starter Kit is a ready-made `.claude/` folder.

What's inside (every file explained)

<code>.claude/</code>	The config folder Claude Code looks for
<code>CLAUDE.md</code>	Project memory - your stack, commands, rules Claude reads this EVERY session. Most important file.
<code>settings.json</code>	Permissions + safety hooks (what Claude can/can't do)
<code>rules/</code>	Conditional rules applied to matching files
<code> testing.md</code>	Auto-applied when working on test files
<code>skills/</code>	Custom /commands you can invoke
<code> commit-msg/</code>	<code>/commit-msg</code> - generates git commit messages
<code> explain-code/</code>	<code>/explain-code</code> - explains code in plain English
<code> code-review/</code>	<code>/code-review</code> - finds bugs & security issues
<code> test-writer/</code>	<code>/test-writer</code> - writes tests for your code
<code>agents/</code>	Subagents - isolated AI workers
<code> reviewer.yml</code>	Code review agent (own context window)
<code>.claudeignore</code>	Files Claude should ignore (like <code>.gitignore</code>)

How the safety hooks work

The `settings.json` includes 2 pre-configured hooks that run automatically:

Hook 1: Dangerous Command Blocker (PreToolUse)

- Runs **before** Claude executes any bash command
- Checks if command contains `rm -rf`, `git push --force`, `DROP TABLE`, etc.
- If dangerous: **exit code 2 = BLOCKED**. Claude cannot proceed. 100% reliable.

Hook 2: Auto-format on Save (PostToolUse)

- Runs **after** Claude writes any `.ts`, `.tsx`, `.js`, or `.jsx` file
- Automatically runs Prettier to format the file
- Your code stays clean without thinking about it

How Skills (slash commands) work

Skills are reusable prompts. Instead of typing the same instructions every time, you type `/name`:

<code>/commit-msg</code>	Stage your changes (git add .), type <code>/commit-msg</code> , get a perfect commit message
<code>/explain-code</code>	Point Claude at any file, type <code>/explain-code</code> , get a plain English breakdown
<code>/code-review</code>	Type <code>/code-review</code> after changes, get a bug/security/improvement report
<code>/test-writer</code>	Point at a file, type <code>/test-writer</code> , get comprehensive tests written and run

CLAUDE CODE - Starter Kit Guide

STEP 1 Unzip the Starter Kit

```
# Unzip the downloaded file
# You'll see: .claude/ folder + .claudeignore + README.md
```

STEP 2 Copy to your project

```
# Mac / Linux
cp -r .claude/ ~/my-project/.claude/
cp .claudeignore ~/my-project/.claudeignore

# Windows (PowerShell)
Copy-Item -Recurse .claude\ ~\my-project\.claude\
Copy-Item .claudeignore ~\my-project\
```

IMPORTANT

Replace ~/my-project with the actual path to YOUR project folder.

STEP 3 Customize CLAUDE.md

Open .claude/CLAUDE.md in any text editor. Replace the placeholders with YOUR project info:

```
## Project Overview
- Stack: Next.js 14 + TypeScript + Prisma <-- YOUR stack
- Description: E-commerce platform <-- YOUR description

## Key Commands
- Build: npm run build <-- YOUR build command
- Test: npm test <-- YOUR test command
- Lint: npm run lint <-- YOUR lint command

## Code Conventions
- TypeScript strict, no any <-- YOUR conventions
- PascalCase components <-- YOUR naming rules

## Architecture
- /src/app -> pages <-- YOUR folder structure
- /src/lib -> business logic <-- YOUR folder structure
```

PRO TIP

Don't overthink it. Start with basics (stack + commands). You can always add more later.
Run /init after setup -- Claude will suggest improvements to your CLAUDE.md.

STEP 4 Launch and verify

```
cd ~/my-project
claude          # Claude loads your .claude/ config!
/help          # You should see your 4 custom skills listed
```

Done! Claude now knows your project, has safety hooks, and 4 custom commands.

Try: git add . then type /commit-msg to see your first skill in action.

CLAUDE CODE - Starter Kit Guide

How to customize & extend

ADD A NEW SKILL

- Create folder: `.claude/skills/your-name/SKILL.md`
- Add frontmatter (name + description) and instructions in Markdown
- Restart Claude Code -- your `/your-name` command is now available

ADD A NEW HOOK

- Edit `.claude/settings.json` -- add to the `hooks` section
- Or type `/hooks` inside Claude Code for interactive setup

ADD A NEW RULE

- Create a `.md` file in `.claude/rules/` -- it applies to matching files

ADD A NEW SUBAGENT

- Create a `.yaml` file in `.claude/agents/` with name, description, model, tools

Frequently Asked Questions

Do I need to edit every file?

No! Only CLAUDE.md is required. Everything else works out of the box.

Will this overwrite my existing .claude/ folder?

If you already have one, merge manually. Don't blindly overwrite.

Does this work with any programming language?

Yes. CLAUDE.md is language-agnostic. Just change the commands and conventions.

What if I don't use Prettier?

Edit settings.json and remove or change the PostToolUse hook.

Can I share this with my team?

Yes! Commit `.claude/` to git. Use settings.local.json for personal overrides (gitignored).

What if a skill doesn't show up?

Check: 1) file is named SKILL.md exactly 2) frontmatter has name + description 3) restart Claude.

How do I remove a skill I don't need?

Delete the skill folder from `.claude/skills/`. That's it.

What's .claudeignore for?

Like `.gitignore` but for Claude. Files listed here won't be loaded into Claude's context.

What to do next

- Run `/init` inside Claude Code -- it will enhance your CLAUDE.md with codebase-specific details
- Try each skill: `/commit-msg`, `/explain-code`, `/code-review`, `/test-writer`
- Create your own skill for a workflow you repeat often